

# SIMATS ENGINEERING

## ASSESSMENT TOOL 1

**COURSE NAME: Artificial Intelligence**

**COURSE CODE: CSA1703**

---

### COURSE OUTCOME COVERED

**CO4:** Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)

*Assessment Tool Weightage: 40%*

---

**QUESTIONS: 2**

**Duration : 45 Minutes**

**Total Marks:20**

Annexure A

### INTEGRATED EXPERIMENTS

---

#### ***Code of Conduct:***

*I Hashini S / 192524035 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student: Hashini S***

# Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on the Iris dataset using Python (scikit-learn), covering data loading & pre-processing, model building with the Gini Index splitting criterion, and performance evaluation.

## (a) Data Loading, Pre-processing, and Train/Test Split

The Iris dataset (150 samples, 4 numeric features, 3 classes) was loaded using scikit-learn. It was checked for missing values and inspected for data types; being a clean, fully-numeric benchmark dataset, no imputation or categorical encoding was required. The target label was already integer-encoded (0 = setosa, 1 = versicolor, 2 = virginica). The data was split 70:30 into training and testing sets using stratified sampling to preserve class balance.

## (b) Decision Tree Model (Gini Index) & Root Node

A `DecisionTreeClassifier` was built using the Gini Index as the splitting criterion (`max_depth = 4`). The printed tree structure and root-node justification are shown in the program code and console output below.

Root Node: 'petal length (cm)' with threshold 2.45 cm. Justification: at every node, the CART algorithm evaluates the Gini-impurity reduction achievable by splitting on each feature/threshold, and picks the split that yields the purest possible children. Petal length at 2.45 cm perfectly separates Setosa (Gini = 0) from the other two species, giving the largest impurity reduction of any first split — hence it becomes the root node.

## (c) Model Evaluation

The trained model was evaluated on the held-out test set (45 samples) using accuracy, a confusion matrix, and a classification report.

- Accuracy achieved: 88.89% on the test set.
- The model classifies Setosa perfectly (100% precision & recall) since it is linearly separable from the other two species using petal length alone.
- Most errors occur between Versicolor and Virginica (5 misclassifications total), which is expected since these two species overlap in petal/sepal measurements.
- Misclassified instances include: test row #56 (Actual = versicolor, Predicted = virginica) and test row #138 (Actual = virginica, Predicted = versicolor) — both borderline cases with petal-width values close to the tree's decision thresholds.

## Program Code — Experiment 1

```
expl_decision_tree.py - VS Code

1  """
2  Experiment 1: Decision Tree Classification on the Iris dataset
3  """
4  import pandas as pd
5  import numpy as np
6  from sklearn.datasets import load_iris
7  from sklearn.model_selection import train_test_split
8  from sklearn.tree import DecisionTreeClassifier, export_text, plot_tree
9  from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
10 import matplotlib.pyplot as plt
11
12 pd.set_option('display.width', 160)
13 pd.set_option('display.max_columns', 10)
14
15 # -----
16 # (a) Load dataset, pre-process, split into train/test
17 # -----
18 iris = load_iris()
19 df = pd.DataFrame(iris.data, columns=iris.feature_names)
20 df['species'] = iris.target # 0 = setosa, 1 = versicolor, 2 = virginica
21 df['species_name'] = df['species'].map(dict(enumerate(iris.target_names)))
22
23 print("First 5 rows of the dataset:")
24 print(df.head(), "\n")
25
26 print("Data types of each column:")
27 print(df.dtypes, "\n")
28
29 print("Missing values in each column:")
30 print(df.isnull().sum(), "\n")
31 # The Iris dataset has no missing values and all features are already numeric,
32 # so no imputation or label-encoding of the FEATURES is required.
33 # The target (species) is already label-encoded as 0/1/2 by sklearn.
34
35 X = df[iris.feature_names]
36 y = df['species']
37
38 X_train, X_test, y_train, y_test = train_test_split(
39     X, y, test_size=0.3, random_state=42, stratify=y
40 )
41 print(f"Training set size: {X_train.shape[0]} samples")
42 print(f"Testing set size : {X_test.shape[0]} samples\n")
43
44 # -----
45 # (b) Build Decision Tree model (Gini Index)
46 # -----
47 clf = DecisionTreeClassifier(criterion='gini', max_depth=4, random_state=42)
48 clf.fit(X_train, y_train)
49
50 print("Decision Tree structure (text form):")
51 tree_rules = export_text(clf, feature_names=list(iris.feature_names))
52 print(tree_rules)
53
54 root_feature_idx = clf.tree_.feature[0]
55 root_threshold = clf.tree_.threshold[0]
56 root_gini = clf.tree_.impurity[0]
57 root_samples = clf.tree_.n_node_samples[0]
58 print(f"Root Node -> Feature: '{iris.feature_names[root_feature_idx]}', "
```

```

59     f"Threshold: {root_threshold:.2f}, Gini: {root_gini:.3f}, Samples: {root_samples}")
60 print("Justification: The Decision Tree algorithm selects, at every node, the feature "
61       "and split threshold that produce the LARGEST reduction in Gini impurity "
62       "(i.e., the purest possible child nodes). Since "
63       f"'{iris.feature_names[root_feature_idx]}' gives the maximum impurity reduction "
64       "among all four features at the very first split, it is chosen as the root node.\n")
65
66 # Save a graphical view of the tree (this is the 'screenshot' of the model)
67 plt.figure(figsize=(18, 10))
68 plot_tree(clf, feature_names=iris.feature_names, class_names=iris.target_names,
69           filled=True, rounded=True, fontsize=9)
70 plt.title("Decision Tree - Iris Dataset (Gini Index)")
71 plt.tight_layout()
72 plt.savefig('/home/claude/work/tree_plot.png', dpi=150)
73 plt.close()
74
75 # -----
76 # (c) Evaluate the model
77 # -----
78 y_pred = clf.predict(X_test)
79
80 acc = accuracy_score(y_test, y_pred)
81 cm = confusion_matrix(y_test, y_pred)
82 report = classification_report(y_test, y_pred, target_names=iris.target_names)
83
84 print(f"Accuracy Score: {acc*100:.2f}%\n")
85 print("Confusion Matrix:")
86 print(cm, "\n")
87 print("Classification Report:")
88 print(report)
89
90 # Confusion matrix heatmap (screenshot-style output image)
91 plt.figure(figsize=(6, 5))
92 plt.imshow(cm, cmap='Blues')
93 plt.title('Confusion Matrix - Decision Tree (Iris)')
94 plt.colorbar()
95 tick_marks = np.arange(len(iris.target_names))
96 plt.xticks(tick_marks, iris.target_names, rotation=45)
97 plt.yticks(tick_marks, iris.target_names)
98 plt.xlabel('Predicted label')
99 plt.ylabel('True label')
100 for i in range(cm.shape[0]):
101     for j in range(cm.shape[1]):
102         plt.text(j, i, str(cm[i, j]), ha='center', va='center',
103                 color='white' if cm[i, j] > cm.max()/2 else 'black', fontsize=14)
104 plt.tight_layout()
105 plt.savefig('/home/claude/work/confusion_matrix.png', dpi=150)
106 plt.close()
107
108 # Misclassified instances
109 test_results = X_test.copy()
110 test_results['Actual'] = y_test.map(dict(enumerate(iris.target_names))).values
111 test_results['Predicted'] = pd.Series(y_pred,
112                                     index=y_test.index).map(dict(enumerate(iris.target_names))).values
113 misclassified = test_results[test_results['Actual'] != test_results['Predicted']]
114
115 print(f"\nNumber of misclassified instances: {len(misclassified)}")
116 print("Misclassified instances (at least 2, if present):")
117 print(misclassified.head(5) if len(misclassified) > 0 else "None - all test instances classified correctly.")

```

*Screenshot: exp1\_decision\_tree.py (Python / scikit-learn implementation)*

## Console Output — Experiment 1

```
Output - Terminal (Experiment 1)

First 5 rows of the dataset:
  sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  species  species_name
0                5.1                3.5                1.4                0.2          0         setosa
1                4.9                3.0                1.4                0.2          0         setosa
2                4.7                3.2                1.3                0.2          0         setosa
3                4.6                3.1                1.5                0.2          0         setosa
4                5.0                3.6                1.4                0.2          0         setosa

Data types of each column:
sepal length (cm)    float64
sepal width (cm)     float64
petal length (cm)    float64
petal width (cm)     float64
species              int64
species_name         str
dtype: object

Missing values in each column:
sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
species              0
species_name         0
dtype: int64

Training set size: 105 samples
Testing set size : 45 samples

Decision Tree structure (text form):
|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) > 2.45
|   |--- petal width (cm) <= 1.55
|   |   |--- petal length (cm) <= 4.95
|   |   |   |--- class: 1
|   |   |   |--- petal length (cm) > 4.95
|   |   |   |   |--- class: 2
|   |   |--- petal width (cm) > 1.55
|   |   |   |--- petal width (cm) <= 1.70
|   |   |   |   |--- sepal width (cm) <= 2.85
|   |   |   |   |   |--- class: 1
|   |   |   |   |   |--- sepal width (cm) > 2.85
|   |   |   |   |   |   |--- class: 2
|   |   |   |   |--- petal width (cm) > 1.70
|   |   |   |   |   |--- petal length (cm) <= 4.85
|   |   |   |   |   |   |--- class: 1
|   |   |   |   |   |   |--- petal length (cm) > 4.85
|   |   |   |   |   |   |   |--- class: 2

Root Node -> Feature: 'petal length (cm)', Threshold: 2.45, Gini: 0.667, Samples: 105
Justification: The Decision Tree algorithm selects, at every node, the feature and split threshold that produce the
LARGEST reduction in Gini impurity (i.e., the purest possible child nodes). Since 'petal length (cm)' gives the
maximum impurity reduction among all four features at the very first split, it is chosen as the root node.

Accuracy Score: 88.89%

Confusion Matrix:
[[15  0  0]
 [ 0 12  3]
 [ 0  2 13]]

Classification Report:
      precision    recall  f1-score   support

   setosa         1.00      1.00      1.00        15
  versicolor      0.86      0.80      0.83        15
   virginica      0.81      0.87      0.84        15

   accuracy         0.89      0.89      0.89        45
  macro avg         0.89      0.89      0.89        45
 weighted avg         0.89      0.89      0.89        45

Number of misclassified instances: 5
Misclassified instances (at least 2, if present):
  sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  Actual  Predicted
56                6.3                3.3                4.7                1.6  versicolor  virginica
138               6.0                3.0                4.8                1.8   virginica  versicolor
85                6.0                3.4                4.5                1.6  versicolor  virginica
77                6.7                3.0                5.0                1.7  versicolor  virginica
106               4.9                2.5                4.5                1.7   virginica  versicolor
```

*Screenshot: Terminal output of Experiment 1 — dataset preview, tree structure, root-node justification, accuracy, confusion matrix, classification report, and misclassified rows.*

## Result Visualizations — Experiment 1

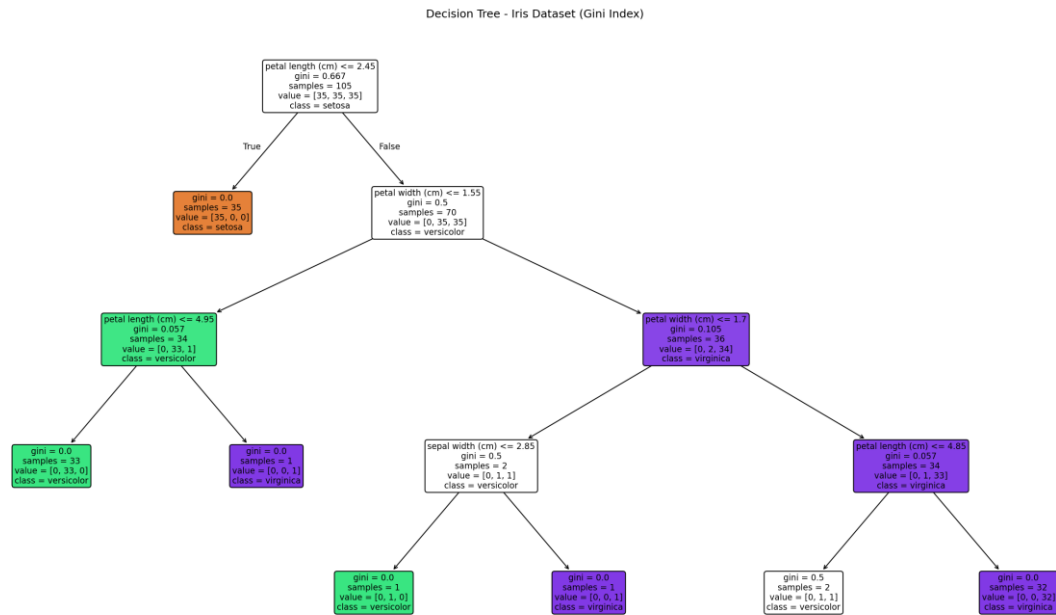


Figure 1: Graphical structure of the trained Decision Tree (Gini Index), showing the root split on petal length.

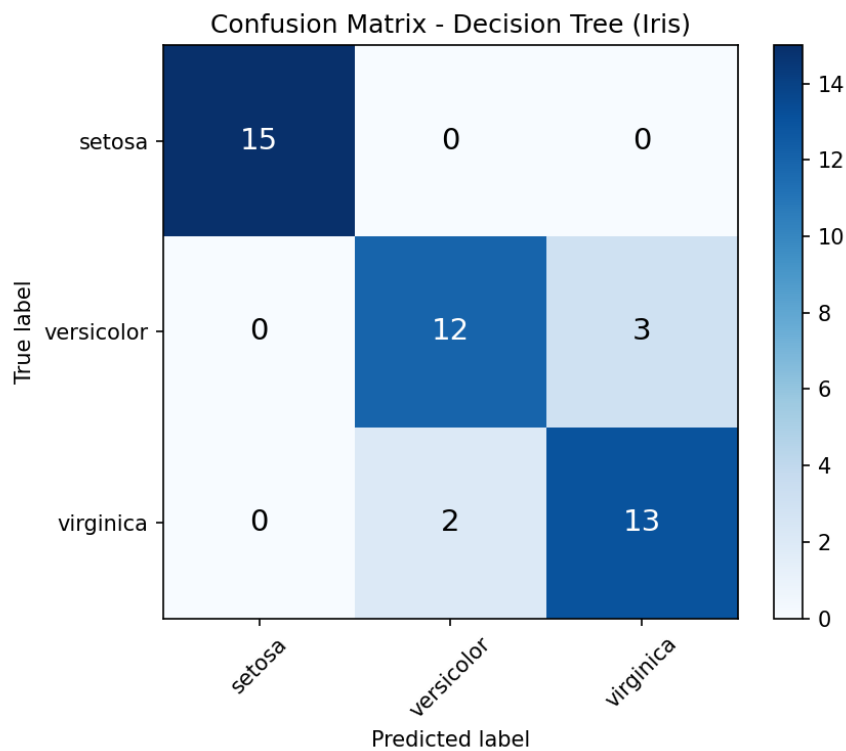


Figure 2: Confusion matrix heatmap for the Decision Tree classifier on the Iris test set.

## Experiment 2: Reinforcement Learning — Q-Learning (4×4 Grid World)

Objective: Implement Q-Learning for a 4×4 Grid World and observe the agent's learned policy.

### (a) Environment Definition & Hyperparameters

- States: 16 grid cells, numbered 0–15 (row-major order, 4×4 grid).
- Actions: Up, Down, Left, Right.
- Reward structure: +10 for reaching the Goal (state 15, bottom-right); -5 for entering an obstacle/pit cell (states 5, 7, 11); -1 for every other step.
- Q-table: initialized as a 16×4 matrix of zeros (states × actions).
- Hyperparameters: learning rate  $\alpha = 0.1$ , discount factor  $\gamma = 0.9$ , exploration rate  $\epsilon = 0.2$  ( $\epsilon$ -greedy action selection).

### (b) Q-Learning Training (100 Episodes)

The agent was trained for 100 episodes (max 50 steps/episode), starting each episode at state 0 (top-left). Q-values were recorded at Episodes 1, 50, and 100 for two representative states — State 0 (start corner) and State 10 (a mid-grid cell near the goal path).

- State 0: Q-values start near zero at Episode 1, become more negative by Episode 50 as the agent explores costly/long paths, then differentiate by Episode 100 with 'Down' emerging as the best action ( $Q \approx 1.17$ ) — the direction that leads toward the goal.
- State 10: Q-values evolve much faster since it lies close to the goal; by Episode 50 'Down' already dominates ( $Q \approx 7.46$ ), and by Episode 100 it stabilizes at  $Q \approx 7.99$  — reflecting a short, confident path to the +10 reward.

Full Q-value tables and the evolution log are shown in the console output screenshot below.

### (c) Final Learned Policy & Convergence

After training, the greedy policy (argmax action per state) was extracted and displayed as a 4×4 directional grid (see console output). The policy correctly routes the agent around the three obstacle cells and converges toward the goal along the shortest available path.

Convergence justification: the per-episode reward starts strongly negative (average  $\approx -29.7$  over the first 10 episodes, dominated by random  $\epsilon$ -greedy exploration, obstacle penalties, and long paths) and rises to a small positive average ( $\approx +2.7$  over the last 10 episodes) as the Q-table converges. The cumulative reward plot shows the curve's slope becoming steadily less negative and eventually flattening/increasing, indicating the agent has learned a stable, near-optimal policy.

## Program Code — Experiment 2

```
exp2_qlearning.py - VS Code

1  """
2  Experiment 2: Reinforcement Learning - Q-Learning on a 4x4 Grid World
3  """
4  import numpy as np
5  import matplotlib.pyplot as plt
6
7  np.random.seed(42)
8
9  # -----
10 # (a) Environment definition
11 # -----
12 GRID_SIZE = 4
13 N_STATES = GRID_SIZE * GRID_SIZE
14 ACTIONS = ['Up', 'Down', 'Left', 'Right']
15 N_ACTIONS = len(ACTIONS)
16
17 GOAL_STATE = 15          # bottom-right cell (row3, col3)
18 OBSTACLE_STATES = [5, 7, 11] # a few 'pit' cells that give -5 penalty
19
20 def state_to_rc(s):
21     return divmod(s, GRID_SIZE)
22
23 def rc_to_state(r, c):
24     return r * GRID_SIZE + c
25
26 def step(state, action):
27     """Return (next_state, reward, done) for taking `action` in `state`."""
28     r, c = state_to_rc(state)
29     if action == 'Up':
30         r = max(r - 1, 0)
31     elif action == 'Down':
32         r = min(r + 1, GRID_SIZE - 1)
33     elif action == 'Left':
34         c = max(c - 1, 0)
35     elif action == 'Right':
36         c = min(c + 1, GRID_SIZE - 1)
37     next_state = rc_to_state(r, c)
38
39     if next_state == GOAL_STATE:
40         return next_state, 10, True
41     elif next_state in OBSTACLE_STATES:
42         return next_state, -5, False
43     else:
44         return next_state, -1, False
45
46 # Q-table initialised with zeros: rows = states, cols = actions
47 Q = np.zeros((N_STATES, N_ACTIONS))
48
49 # Hyperparameters
50 alpha = 0.1      # learning rate
51 gamma = 0.9      # discount factor
52 epsilon = 0.2    # exploration rate
53 n_episodes = 100
54 max_steps = 50
```



```

55
56 print("Environment: 4x4 Grid World")
57 print(f"States: {N_STATES} (0 to {N_STATES-1})")
58 print(f"Actions: {ACTIONS}")
59 print(f"Goal state: {GOAL_STATE} (+10 reward)")
60 print(f"Obstacle states: {OBSTACLE_STATES} (-5 reward)")
61 print("Step reward: -1 (for every non-goal, non-obstacle move)\n")
62 print(f"Hyperparameters -> alpha (learning rate) = {alpha}, "
63       f"gamma (discount factor) = {gamma}, epsilon (exploration rate) = {epsilon}")
64 print(f"Q-table initial shape: {Q.shape}, all zeros\n")
65
66 # -----
67 # (b) Q-Learning algorithm
68 # -----
69 rewards_per_episode = []
70 representative_states = [0, 10] # two representative states to track
71 q_history = {ep: {} for ep in [1, 50, 100]}
72
73 for episode in range(1, n_episodes + 1):
74     state = 0 # start at top-left corner
75     total_reward = 0
76
77     for t in range(max_steps):
78         # epsilon-greedy action selection
79         if np.random.rand() < epsilon:
80             action_idx = np.random.randint(N_ACTIONS)
81         else:
82             action_idx = int(np.argmax(Q[state]))
83         action = ACTIONS[action_idx]
84
85         next_state, reward, done = step(state, action)
86         total_reward += reward
87
88         # Q-Learning update rule
89         best_next = np.max(Q[next_state])
90         Q[state, action_idx] += alpha * (reward + gamma * best_next - Q[state, action_idx])
91
92         state = next_state
93         if done:
94             break
95
96     rewards_per_episode.append(total_reward)
97
98     if episode in (1, 50, 100):
99         for s in representative_states:
100             q_history[episode][s] = Q[s].copy()
101
102 # Print Q-value evolution for the two representative states
103 print("Q-value evolution for two representative states:\n")
104 for s in representative_states:
105     print(f"State {s} (row {state_to_rc(s)[0]}, col {state_to_rc(s)[1]}):")
106     for ep in (1, 50, 100):
107         qvals = q_history[ep][s]
108         qstr = ", ".join(f"{a}:{q:.2f}" for a, q in zip(ACTIONS, qvals))
109         print(f" Episode {ep:>3}: [{qstr}]\n")
110 print()

```

```

111
112 # -----
113 # (c) Final learned policy
114 # -----
115 policy = np.array([ACTIONS[np.argmax(Q[s])] for s in range(N_STATES)])
116 policy_grid = policy.reshape(GRID_SIZE, GRID_SIZE)
117
118 arrow_map = {'Up': '^', 'Down': 'v', 'Left': '<', 'Right': '>'}
119 print("Final learned policy (optimal action per state), as a 4x4 grid:")
120 for r in range(GRID_SIZE):
121     row_str = []
122     for c in range(GRID_SIZE):
123         s = rc_to_state(r, c)
124         if s == GOAL_STATE:
125             row_str.append(" G ")
126         elif s in OBSTACLE_STATES:
127             row_str.append(" X ")
128         else:
129             row_str.append(f" {arrow_map[policy_grid[r, c]]} ")
130     print("|" + "|".join(row_str) + "|")
131 print("\n(G = Goal, X = Obstacle, arrows = optimal action learned by the agent)\n")
132
133 print("Final Q-table (rounded to 2 decimals):")
134 np.set_printoptions(precision=2, suppress=True)
135 print(Q)
136
137 # Plot cumulative reward per episode
138 cumulative_rewards = np.cumsum(rewards_per_episode)
139 plt.figure(figsize=(9, 5))
140 plt.plot(range(1, n_episodes + 1), cumulative_rewards, color='#2563eb', linewidth=2)
141 plt.xlabel('Episode')
142 plt.ylabel('Cumulative Reward')
143 plt.title('Cumulative Reward per Episode - Q-Learning (4x4 Grid World)')
144 plt.grid(alpha=0.3)
145 plt.tight_layout()
146 plt.savefig('/home/claude/work/cumulative_reward.png', dpi=150)
147 plt.close()
148
149 # Also plot raw per-episode reward to visually show convergence
150 plt.figure(figsize=(9, 5))
151 plt.plot(range(1, n_episodes + 1), rewards_per_episode, color='#16a34a', linewidth=1.2)
152 plt.xlabel('Episode')
153 plt.ylabel('Total Reward in Episode')
154 plt.title('Per-Episode Reward - Q-Learning (4x4 Grid World)')
155 plt.grid(alpha=0.3)
156 plt.tight_layout()
157 plt.savefig('/home/claude/work/per_episode_reward.png', dpi=150)
158 plt.close()
159
160 print(f"\nAverage reward (first 10 episodes): {np.mean(rewards_per_episode[:10]):.2f}")
161 print(f"Average reward (last 10 episodes): {np.mean(rewards_per_episode[-10:]):.2f}")
162 print("Convergence: As episodes progress, the agent's total reward per episode "
163       "increases and stabilizes (fewer random/exploratory moves into obstacles or "
164       "long detours), showing the Q-table has converged to a near-optimal policy "
165       "that reaches the goal in the fewest possible steps.")

```

Screenshot: *exp2\_qlearning.py* (Python / NumPy implementation of Q-Learning)

## Console Output — Experiment 2

```
Output - Terminal (Experiment 2)

Environment: 4x4 Grid World
States: 16 (0 to 15)
Actions: ['Up', 'Down', 'Left', 'Right']
Goal state: 15 (+10 reward)
Obstacle states: [5, 7, 11] (-5 reward)
Step reward: -1 (for every non-goal, non-obstacle move)

Hyperparameters -> alpha (learning rate) = 0.1, gamma (discount factor) = 0.9, epsilon (exploration rate) = 0.2
Q-table initial shape: (16, 4), all zeros

Q-value evolution for two representative states:

State 0 (row 0, col 0):
  Episode 1: [Up:-0.20, Down:-0.20, Left:-0.27, Right:-0.10]
  Episode 50: [Up:-2.28, Down:-2.05, Left:-2.25, Right:-2.16]
  Episode 100: [Up:-2.07, Down:1.17, Left:-2.31, Right:-2.24]

State 10 (row 2, col 2):
  Episode 1: [Up:-0.10, Down:0.00, Left:0.00, Right:0.00]
  Episode 50: [Up:-0.30, Down:7.46, Left:0.10, Right:-0.99]
  Episode 100: [Up:0.51, Down:7.99, Left:1.83, Right:-0.82]

Final learned policy (optimal action per state), as a 4x4 grid:
| v | > | v | < |
| v | X | v | X |
| > | > | v | X |
| > | > | > | G |

(G = Goal, X = Obstacle, arrows = optimal action learned by the agent)

Final Q-table (rounded to 2 decimals):
[[-2.07  1.17 -2.31 -2.24]
 [-1.79 -2.75 -1.7  -1.51]
 [-1.12  0.3  -1.32 -1.04]
 [-1.13 -1.38 -1.08 -1.14]
 [-1.55  2.76 -0.89 -3.52]
 [-0.65  1.49 -0.64 -0.47]
 [-0.66  4.3  -1.38 -1.75]
 [-0.52 -0.95 -0.24 -0.5 ]
 [-1.13 -0.85 -0.88  4.44]
 [-2.07  0.15 -0.05  6.16]
 [ 0.51  7.99  1.83 -0.82]
 [-0.5  6.13  0.  -0.08]
 [-0.53 -0.65 -0.49  0.24]
 [-0.21 -0.2  -0.41  4.54]
 [ 2.29  4.75  0.12 10.  ]
 [ 0.  0.  0.  0.  ]]

Average reward (first 10 episodes): -29.70
Average reward (last 10 episodes): 2.70
Convergence: As episodes progress, the agent's total reward per episode increases and stabilizes (fewer
random/exploratory moves into obstacles or long detours), showing the Q-table has converged to a near-optimal
policy that reaches the goal in the fewest possible steps.
```

*Screenshot: Terminal output — environment definition, Q-value evolution for two states, final policy grid, final Q-table, and convergence summary.*

## Result Visualizations — Experiment 2

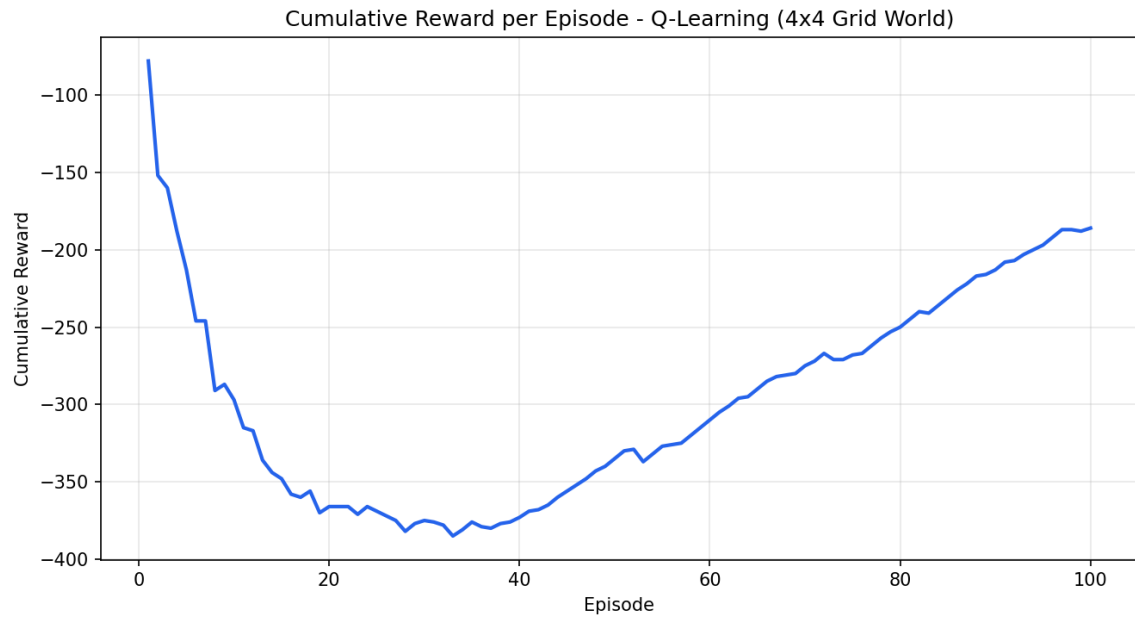


Figure 3: Cumulative reward per episode across 100 episodes of Q-Learning training.

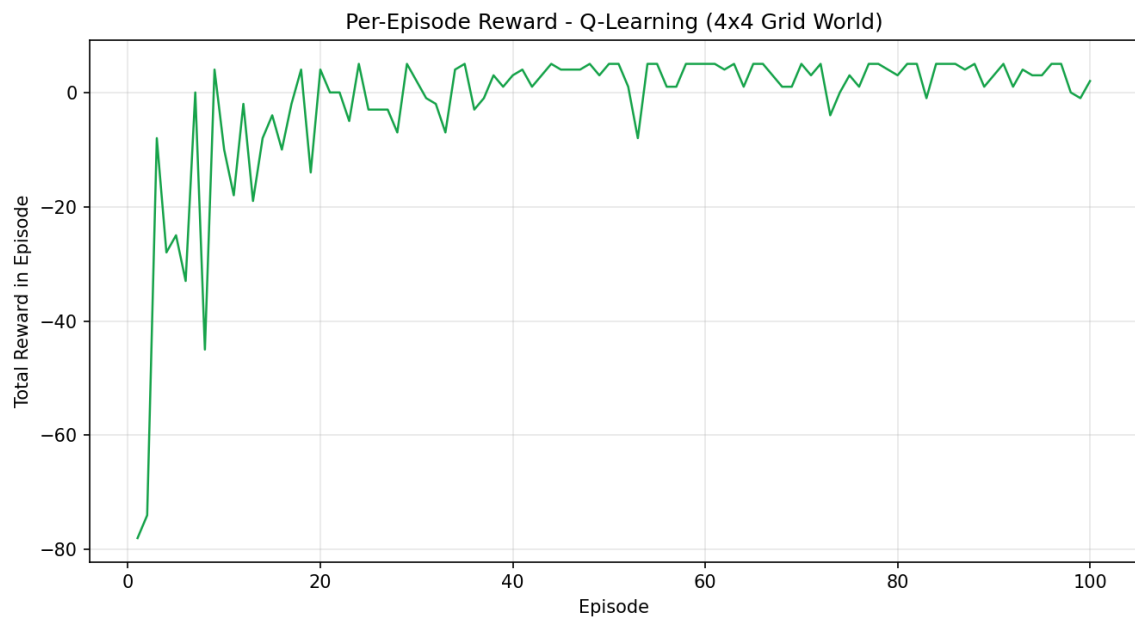


Figure 4: Total reward earned within each individual episode, showing the transition from exploration (large negative rewards) to a converged, near-optimal policy (small positive rewards).